

Simulated Factory Assembly Line

Student Edition — VEX EXP · C++ in VS Code

Name: _____ Date: _____ Period: _____

Introduction

The capstone of the unit — and it takes the whole class. Your team builds one **workcell**: an automated station that performs one process on a part. You'll program the **cell cycle** (wait for the part, run your process, get ready), keep your code tidy with a **function**, and then tackle the real challenge: **handing the part off** to the teams before and after you, so the whole line runs as one system. Everything you've built this year comes together here. Read Chapter 12 in the textbook for the full explanation.

Learning Objectives

- Explain how an assembly line is a system of coordinated automated workcells — the Chapter 8 systems idea, one level up.
- Program a workcell cycle: a sensor-triggered while (true) loop that runs a process each time a part arrives.
- Organize a program with a function — define a named group of commands, then call it — connecting to decomposition.
- Coordinate hand-offs with neighboring cells, and build and integrate your cell as a team.

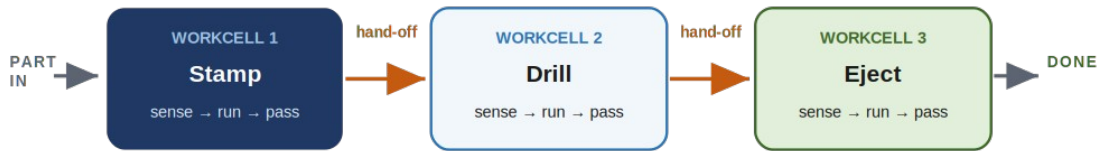
Key Ideas (quick reference)

- **An assembly line is a system of cells:** a part flows through; each cell does one process and passes it on; no single robot makes the whole product. The result comes from cells cooperating.
- **A workcell runs a cycle:** while (true) { if a part has arrived (sensor), run the process, then wait so it passes on }. It is Chapter 11's loop, with the sensor used as a 'has the part arrived?' trigger.
- **Functions keep code readable:** define a named group of commands once (above main), then call it by writing its name. Tuck your process into runProcess() and the loop reads like plain English. It's decomposition from Chapter 9 — a function is a name for one simple behavior.
- **Coordination is the hard part:** neighboring teams must agree on the interface — where the part arrives and leaves, how it sits, and when each cell runs. This is systems integration; one bad hand-off stalls the whole line.

One Part, Many Cells

ONE PART, MANY CELLS — A SIMULATED ASSEMBLY LINE

each team builds one automated workcell; together the cells hand the part down the line



Each cell is a Chapter 11 automatic behavior. The new challenge is making them work together.

The orange hand-offs are where teams must agree: where the part arrives, how it sits, and when to run.

A part flows through the line. Each workcell senses the part, runs its one process, and hands it on. The orange hand-offs are the coordination points where your team and your neighbors must agree.

Your Workcell, in Code

Each cell waits for a part, runs its process, and gets ready for the next — Chapter 11's while (true) loop, with the sensor as a trigger. The process steps live in a **function** so the loop stays readable:

A complete workcell – it stamps each part that arrives

```
#include "vex.h"
using namespace vex;

motor stampArm      = motor(PORT1);
distance partSensor = distance(PORT2);

void runProcess() {           // our cell's job: stamp the part
    stampArm.spin(forward);    // press down
    wait(1, seconds);
    stampArm.spin(reverse);    // lift up
    wait(1, seconds);
    stampArm.stop();
}

int main() {
    vexcodeInit();
    stampArm.setVelocity(40, percent);

    while (true) {             // the workcell cycle
        if (partSensor.objectDistance(mm) < 80) { // a part arrived
            runProcess();
            wait(1, seconds);
        }
        wait(20, msec);
    }
}
```

Two new moves: **runProcess()** is a **function** — define it once above main, call it by name inside the loop (it's decomposition from Chapter 9). And the **while (true)** loop uses the sensor as a 'has the part arrived?' check, with a short wait after so the part can move on. Swap in your own process and sensor to build your cell.

Build Constraints

Build constraints (still in force). (1) Build your workcell from the classroom PLTW Gateway kit

only — the competition team's V5 parts stay separate. (2) Your cell must fit one IKEA Kallax cubby — a 33 × 33 cm (13 × 13 in) opening, about 38 cm (15 in) deep — so it stores neatly and the cells are a consistent size when you line them up.

Safety

- A running line has many moving cells at once. Keep hands, hair, and sleeves clear of every mechanism, not just your own.
- Agree on a way to stop the whole line fast if something jams, and make sure everyone knows it before the first run.
- Secure each cell so a moving part can't pinch fingers or fling pieces; run driving programs only after unplugging the USB-C cable.
- Power Brains down with the X button and store each cell in its cubby when finished.

Equipment & Materials

- Your computer with VS Code and the VEX extension, and a USB-C cable
- The PLTW Gateway EXP kit: Brain, charged battery, motor(s), and a sensor for your process
- The shared part the class is 'manufacturing,' and an idea of the cells before and after yours
- Your engineering notebook and this project worksheet (design brief, interface plan, journal, reflection)

Procedure

1. **Understand your cell — and its neighbors.** Learn your process, then meet the teams before and after you. Agree on the hand-off (Worksheet 2): where the part arrives and leaves, and how it sits.
2. **Sketch and build.** Sketch a couple of designs, pick one, and build your workcell from the kit so it fits the cubby. Mount and wire the motor(s) and the sensor.
3. **Plan, then code.** Write pseudocode for the cell cycle (Worksheet 3) — wait for the part, run the process, get ready. Then write it in C++: put your process steps in a function, called inside a while (true) loop.
4. **Test your cell alone.** Download and run until your cell does its job reliably, every time, by itself. Tune the sensor trigger and motor timing.
5. **Integrate.** Join your cell to your neighbors'. Run a part across the hand-offs and fix what doesn't line up — position, orientation, timing.
6. **Run the line, then evaluate.** With the whole class, run a part from one end to the other. Keep your daily journal (Worksheet 4) and answer the conclusion questions.

Worksheet 1 — Design Brief

Define your cell's job before you build.

Section	Your answer
Your cell's process (what it does to the part)	
The part — what is it, and what state is it in?	
Sensor that tells you a part has arrived	

Section	Your answer
Constraints (kit only, fits cubby)	

Worksheet 2 — Interface / Hand-off Plan

Fill this in **with the teams on either side of you**. The part has to flow smoothly across every hand-off, so agree on the details now — not on run day.

Agree on...	Cell BEFORE me (I receive)	Cell AFTER me (I pass on)
What is the part / its state?		
Where does it arrive / leave?		
How is it sitting (orientation, height)?		
When does the hand-off happen?		

Worksheet 3 — Plan the Code

Plan your cell cycle in plain language **before** you write C++. Put your process steps in runProcess(), and call it in the loop.

Pseudocode

```

/*
  define runProcess():
  {
    _____
    _____
  }

  while (true)
  {
    if ( a part has arrived )
    {
      runProcess()
    }
  }
*/

```

Worksheet 4 — Daily Journal

A couple of sentences per day — especially what got in your way and how you fixed it.

Day	What we did / obstacles we hit and how we solved them
1	

Day	What we did / obstacles we hit and how we solved them
2	
3	

Conclusion Questions

Answer in complete sentences.

1. What would you have done to improve your workcell?
2. What was your team's greatest challenge with its workcell, and how did you handle it?
3. What did you do to contribute to your team's success?
4. Is there anything you could have done differently to help your entire class succeed with the line?
5. What makes the assembly line a system? How is it like — and unlike — the single robot-as-a-system from Chapter 8?
6. What is a function, and why does putting your process in one make a program easier to read?